

Arquitectura de estados

Aplicación en Unity para el Proyecto Cenigraf

Diseño, flujo de datos, contratos y ruta de implementación

Motor: Unity 6 (6000.3.18f1)

Lenguaje: C#

Documento: Versión 1.0

Fecha: Agosto de 2026

Índice

1. Propósito y alcance	1
1.1. Alcance funcional	1
1.2. Convenciones de estado de avance	1
1.3. Vista general de la composición	1
2. Patrón arquitectónico	2
2.1. Componentes y responsabilidades	3
2.2. Flujo unidireccional	4
2.3. Propiedades del diseño	4
2.4. Inmutabilidad práctica	5
3. Composición de los estados	5
3.1. Estado principal	5
3.2. WorldStateData	6
3.3. PlayerStateData	7
3.4. ObjDecorationStateData	9
4. Aplicación del flujo en Unity	12
4.1. Inicialización	12
4.2. Movimiento del jugador	12
4.3. Aplicación a componentes de runtime	13
4.4. Eventos y prevención de ciclos	13
4.5. Persistencia y cambio de escena	13
5. Ruta de implementación propuesta	14
5.1. Etapa 1: consolidar contratos y nombres	14
5.2. Etapa 2: completar PlayerState	14
5.3. Etapa 3: implementar WorldState	14
5.4. Etapa 4: implementar decoraciones como colección	15
5.5. Etapa 5: fachada de estado principal	15
5.6. Mapa completo de referencia	16
6. Validación, pruebas y observabilidad	18

6.1. Invariantes por dominio	18
6.2. Pirámide de pruebas	18
6.2.1. Pruebas unitarias de reductores	18
6.2.2. Pruebas de stores	18
6.2.3. Pruebas de integración en Play Mode	18
6.3. Registro de transiciones	19
6.4. Rendimiento	19
6.5. Criterios de aceptación de la arquitectura	19
7. Anexos	20
7.1. Correspondencia con el código actual	20
7.2. Glosario	20
7.3. Decisiones pendientes	21
7.4. Registro de fuentes visuales	21

Índice de figuras

1.	Composición planeada del estado principal, el mundo, el jugador y los objetos decorativos.	2
2.	Flujo conceptual desde el estado actual hasta la actualización del store y la reconciliación física.	4
3.	Estructura de datos y operaciones propuestas para el estado World.	7
4.	Estructura planeada del jugador y catálogo conceptual de operaciones.	9
5.	Estructura planeada de ObjDecoration y operaciones de actualización.	11
6.	Mapa consolidado de estados, propiedades y operaciones planeadas.	17

1 Propósito y alcance

Este documento define la arquitectura de estados del Proyecto Cenigraf y su aplicación en Unity. Su objetivo es establecer un lenguaje común entre diseño, programación y pruebas para representar el mundo, el jugador y los objetos decorativos mediante datos explícitos y transiciones controladas.

La propuesta toma como fuente los diagramas entregados y contrasta sus conceptos con el código C# disponible en el repositorio. Esta distinción es importante: los diagramas describen la visión objetivo, mientras que las clases del proyecto muestran qué partes ya funcionan en Unity.

1.1 Alcance funcional

La documentación cubre:

- composición del estado principal y de los estados `World`, `Player` y `ObjDecoration`;
- responsabilidades de `StateData`, `Action`, `Payload`, `Reducer`, `Store` y adaptadores de runtime;
- flujo de movimiento y reconciliación con `Rigidbody2D`;
- correspondencia entre los campos planeados y tipos apropiados de Unity/C#;
- estrategia incremental para completar los estados aún no implementados;
- reglas de validación, persistencia y pruebas.

No se describe aquí la lógica de interfaz, autenticación o publicación de eventos, excepto cuando consumen o producen cambios de estado.

1.2 Convenciones de estado de avance

Cuadro 1: Estados usados para describir el avance técnico

Etiqueta	Significado
Implementado	Existe una representación funcional en el código actual.
Parcial	Existe la infraestructura base, pero el modelo no contiene todavía todos los campos del diseño.
Planeado	Está definido en los diagramas o en esta especificación, pero requiere implementación.

1.3 Vista general de la composición

El estado principal se concibe como un contenedor de subestados independientes. Esta división evita que una actualización del jugador tenga que reconstruir o conocer todos los detalles del mundo, y permite evolucionar cada dominio con acciones y reductores propios.

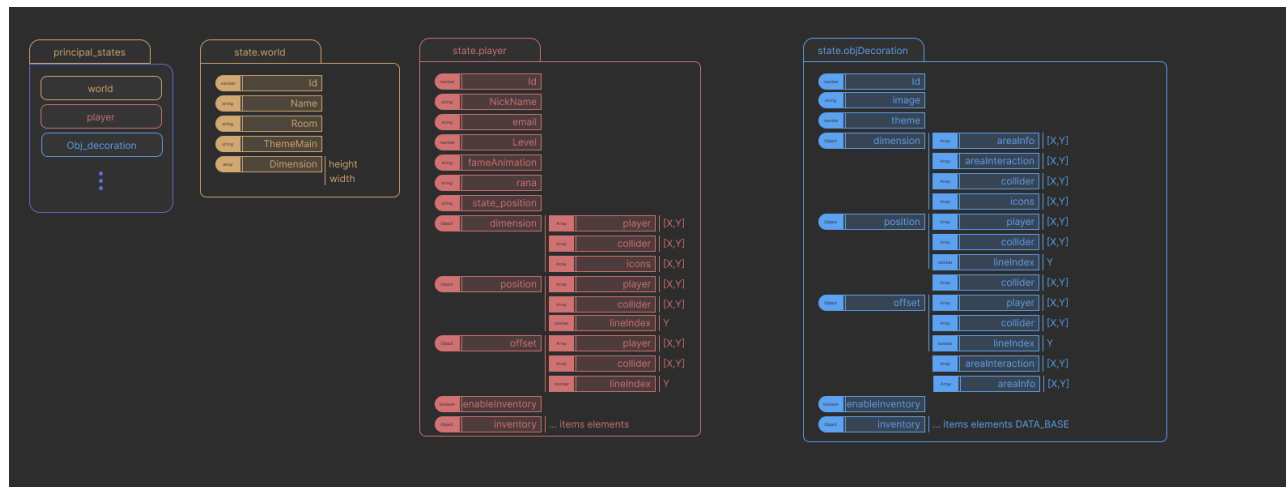


Figura 1: Composición planeada del estado principal, el mundo, el jugador y los objetos decorativos.

La Figura 1 funciona como mapa conceptual, no como definición literal de tipos C#. En las secciones siguientes se normalizan nombres, tipos y responsabilidades para que el diseño sea compatible con serialización, física 2D y convenciones de Unity.

2 Patrón arquitectónico

La arquitectura adopta un flujo unidireccional inspirado en Redux, ajustado al ciclo de vida de Unity. Los datos actuales residen en un *store*; cualquier intención de cambio se expresa como una acción; un reductor calcula el siguiente estado; finalmente, un adaptador aplica al runtime los efectos necesarios.

2.1 Componentes y responsabilidades

Cuadro 2: Responsabilidades de los componentes del patrón

Componente	Responsabilidad	Restricción principal
StateData	Contener una instantánea serializable del dominio.	No controla objetos de escena ni ejecuta efectos.
Action	Nombrar una intención: mover, cambiar rol, configurar colisión.	Debe ser explícita y tener un tipo inequívoco.
Payload	Transportar los valores necesarios para la acción.	Solo datos; sin acceso al store.
Reducer	Calcular el siguiente estado desde el estado actual y la acción.	Debe ser determinista y no modificar la escena.
Store	Custodiar el estado, ejecutar el reductor y publicar cambios.	Es el único punto de escritura del estado.
Runtime Applier	Traducir el estado a componentes Unity.	Escucha al store; no decide reglas de dominio.
Controlador	Lee entrada o eventos y despacha acciones.	No escribe directamente el StateData.

2.2 Flujo unidireccional

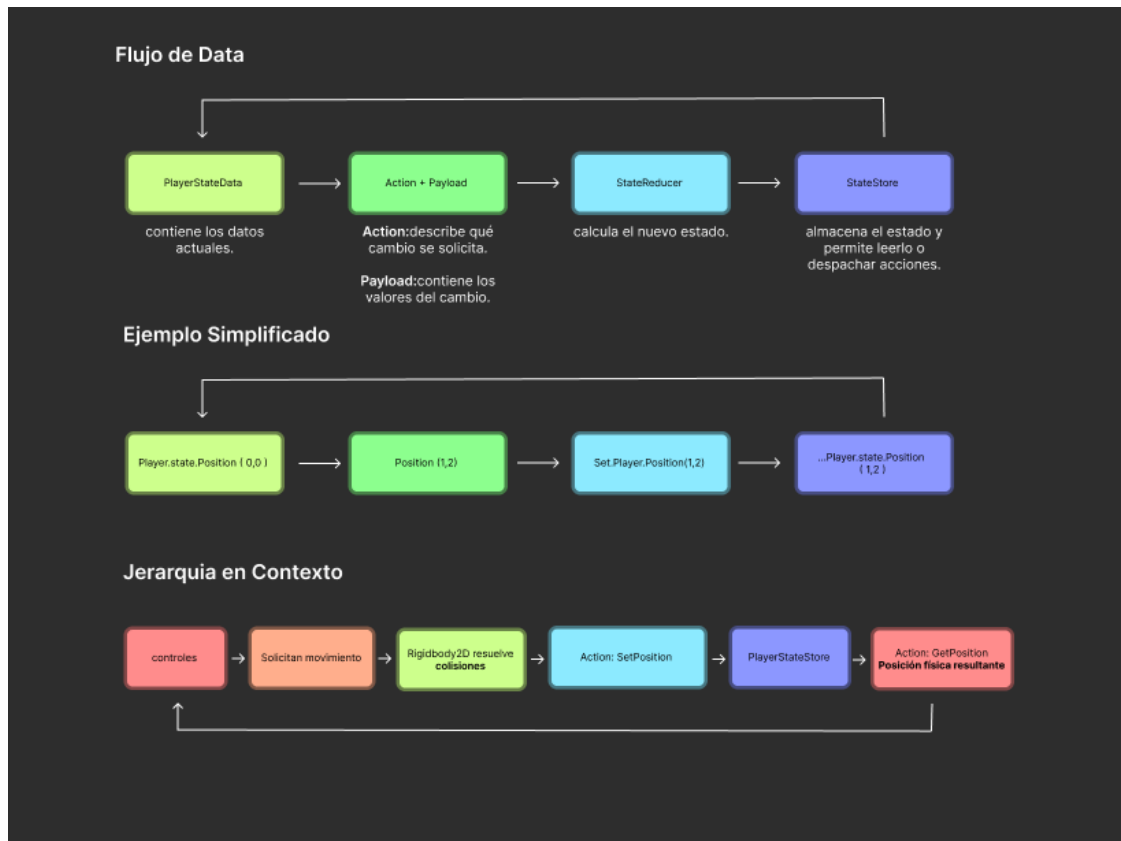


Figura 2: Flujo conceptual desde el estado actual hasta la actualización del store y la reconciliación física.

El flujo esperado es:

1. un controlador captura una intención del usuario o un evento del juego;
2. calcula o solicita un valor permitido, por ejemplo una posición sin penetrar colisiones;
3. crea una **Action** con su **Payload**;
4. el **Store.Dispatch** entrega la acción al **Reducer**;
5. el reductor valida reglas simples y devuelve una nueva instantánea;
6. si el estado cambió, el store publica **StateChanged**;
7. los adaptadores actualizan **Rigidbody2D**, **BoxCollider2D**, renderizadores u otros componentes;
8. cuando Unity altera el valor final por física, se despacha una acción de reconciliación.

2.3 Propiedades del diseño

- **Trazabilidad:** cada mutación se identifica mediante un tipo de acción.

- **Repetibilidad:** el mismo estado y la misma acción producen el mismo resultado.
- **Desacoplamiento:** la lógica de datos no necesita referencias a objetos de escena.
- **Pruebas simples:** reductores y constructores se prueban sin cargar una escena.
- **Integración segura:** los efectos de Unity quedan concentrados en controladores y adaptadores.

2.4 Inmutabilidad práctica

En el código actual, `PlayerStateData` y `GameSessionStateData` son estructuras serializables con campos privados y propiedades de solo lectura. Los métodos `With...` construyen una nueva instantánea en vez de exponer setters. Este enfoque debe mantenerse para los estados planeados.

Listing 1: Forma recomendada de producir una nueva instantánea

```
1 public WorldStateData WithRoom(string value) =>
2     new WorldStateData(id, name, value, mainTheme, dimensions);
```

El uso de métodos `set.X` y `get.X` en los diagramas representa la intención funcional. En C# se recomienda expresarla mediante fábricas de acciones, propiedades de solo lectura y métodos `With...`; no mediante setters públicos que permitan saltarse el store.

3 Composición de los estados

3.1 Estado principal

El diseño plantea un agregado raíz con los subestados `world`, `player` y una colección de `objDecoration`. En el proyecto actual no existe todavía un único `PrincipalStateData`; hay stores separados para sesión, jugador y cámara. Esta separación ya ofrece independencia por dominio y puede conservarse, incorporando un coordinador solamente si aparecen operaciones que necesiten una instantánea global consistente.

Cuadro 3: Mapa de dominios y avance actual

Dominio	Estado	Evidencia en el proyecto
Sesión	Implementado	<code>GameSessionStateData</code> , acciones, reductor y store persistente entre escenas.
Jugador	Parcial	Posición, rol, ordenamiento Y y configuración del collider; faltan identidad, apariencia e inventario del diseño.
Cámara	Implementado	Posición, objetivo, retardo, suavizado, ajuste a píxel y zoom.
Mundo	Planeado	Existe lógica de renderizado por Y, pero no un <code>WorldStateData</code> ni su store.
ObjDecoration	Planeado	Los diagramas definen sus datos; no hay un store/reductor específico localizado.
Estado principal	Planeado/opcional	Puede materializarse como fachada de stores o como agregado serializable para guardado.

3.2 WorldStateData

El estado del mundo identifica el escenario activo y sus dimensiones lógicas. No debe almacenar referencias directas a escenas o GameObjects si se pretende persistir o sincronizar.

Cuadro 4: Campos planeados para el estado del mundo

Campo	Tipo recomendado	Descripción
id	string	Identificador estable del mundo; se prefiere texto sobre un número dependiente de base de datos.
name	string	Nombre visible o administrativo.
room	string	Sala, zona o escena lógica activa.
mainTheme	string	Identificador del tema visual principal.
dimensions	Vector2Int	Ancho y alto del mapa en celdas o unidades lógicas.

Las acciones mínimas serían `SetWorld`, `SetRoom`, `SetMainTheme` y `SetDimensions`. Para cargar un mundo completo conviene una sola acción con un payload compuesto, evitando que los consumidores observen configuraciones intermedias incoherentes.

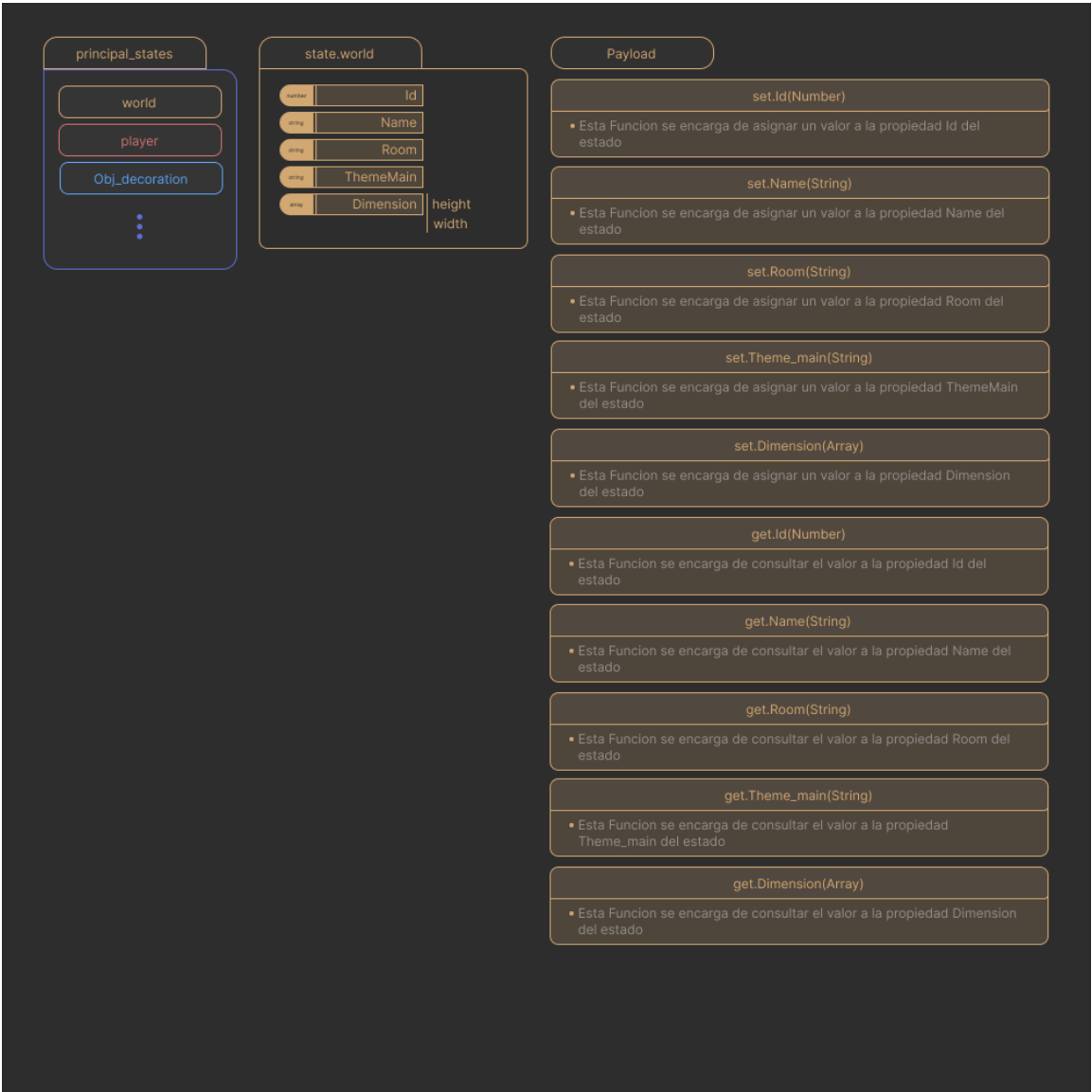


Figura 3: Estructura de datos y operaciones propuestas para el estado World.

3.3 PlayerStateData

El jugador tiene una implementación parcial que ya controla los valores con efecto inmediato en física y renderizado. El modelo objetivo agrega identidad, progresión, apariencia e inventario.

Cuadro 5: Campos del jugador: diseño y normalización recomendada

Campo	Tipo recomendado	Propósito y estado
id	string	Identidad estable. Planeado.
nickname	string	Nombre público. Planeado.

Campo	Tipo recomendado	Propósito y estado
email	string	Dato de cuenta; no debería viajar en estados públicos de red. Planeado.
level	int	Progresión; validar un mínimo de cero o uno. Planeado.
frameAnimation	int	Índice de cuadro si realmente es dato persistente; de lo contrario pertenece al runtime. Planeado.
skinId	string	Identificador de apariencia. El diagrama usa rana/skin ; se recomienda un nombre único. Planeado.
roleId	string	Rol elegido en la sesión. Implementado.
position	Vector2	Posición lógica del cuerpo 2D. Implementado.
colliderSize	Vector2	Dimensiones del BoxCollider2D. Implementado con mínimo de 0.01.
colliderOffset	Vector2	Desplazamiento del collider respecto al transform. Implementado.
ySortEnabled	bool	Activa el ordenamiento visual por coordenada Y. Implementado.
dimensions	PlayerSpatialData	Agrupación opcional para sprites, colliders e iconos; planeada y por definir.
inventoryEnabled	bool	Permite activar el subsistema de inventario. Planeado.
inventory	InventoryStateData	Lista o agregado de ítems identificables. Planeado.

Los bloques **dimension**, **position** y **offset** del diagrama mezclan matrices, valores escalares e información para varios sistemas. En Unity conviene separar semánticas: **Vector2** para medidas/posiciones, arreglos tipados para datos por capa y estructuras específicas cuando un bloque crece. Una matriz **[X,Y]** solo es apropiada si representa una cuadrícula real.

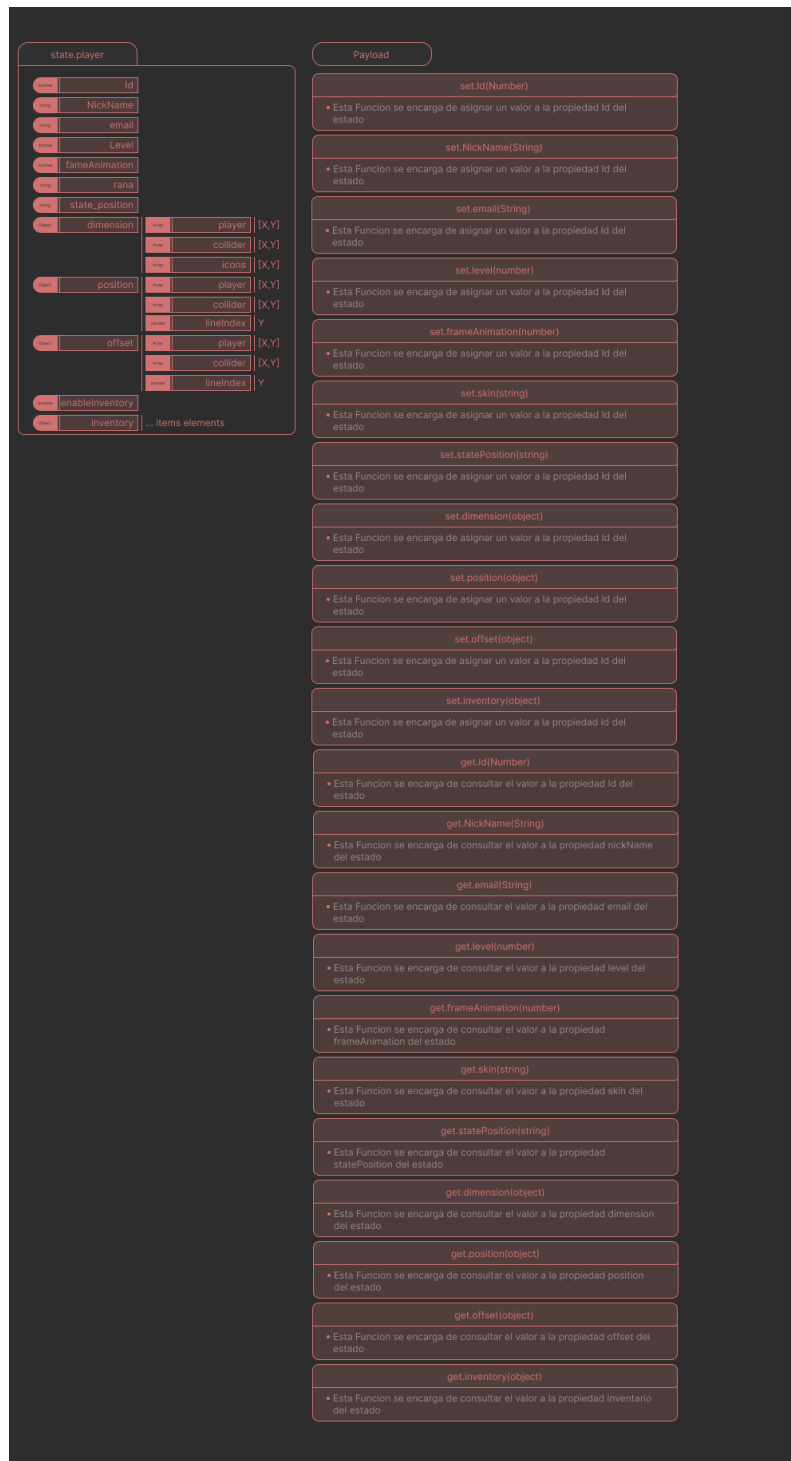


Figura 4: Estructura planeada del jugador y catálogo conceptual de operaciones.

3.4 ObjDecorationStateData

Un objeto decorativo describe elementos del escenario que pueden tener representación visual, colisión, interacción o inventario. Para mantener el sistema escalable, cada instancia necesita identidad propia y los objetos repetidos deben almacenarse en una colección indexada por **id**.

Cuadro 6: Campos planeados para objetos decorativos

Campo	Tipo recomendado	Descripción
id	string	Identificador único y estable.
imageId	string	Clave de sprite, Addressable o recurso; evita serializar un Sprite dentro del estado puro.
themeId	string	Tema o variante visual.
position	Vector2	Posición lógica en el mundo.
dimensions	Vector2	Tamaño visual o de celda.
colliderSize	Vector2	Tamaño del collider, separado del sprite.
colliderOffset	Vector2	Desplazamiento local del collider.
lineIndex	int	Índice de línea/capa si el mapa lo requiere; para Y-sort puede derivarse de la posición.
areaInfo	AreaInfoState[]	Regiones informativas.
interactions	InteractionState[]	Interacciones disponibles.
icons	string[]	Identificadores de iconos asociados.
inventoryEnabled	bool	Controla si el objeto puede contener ítems.
inventory	InventoryStateData	Contenido del objeto; preferiblemente IDs y cantidades.

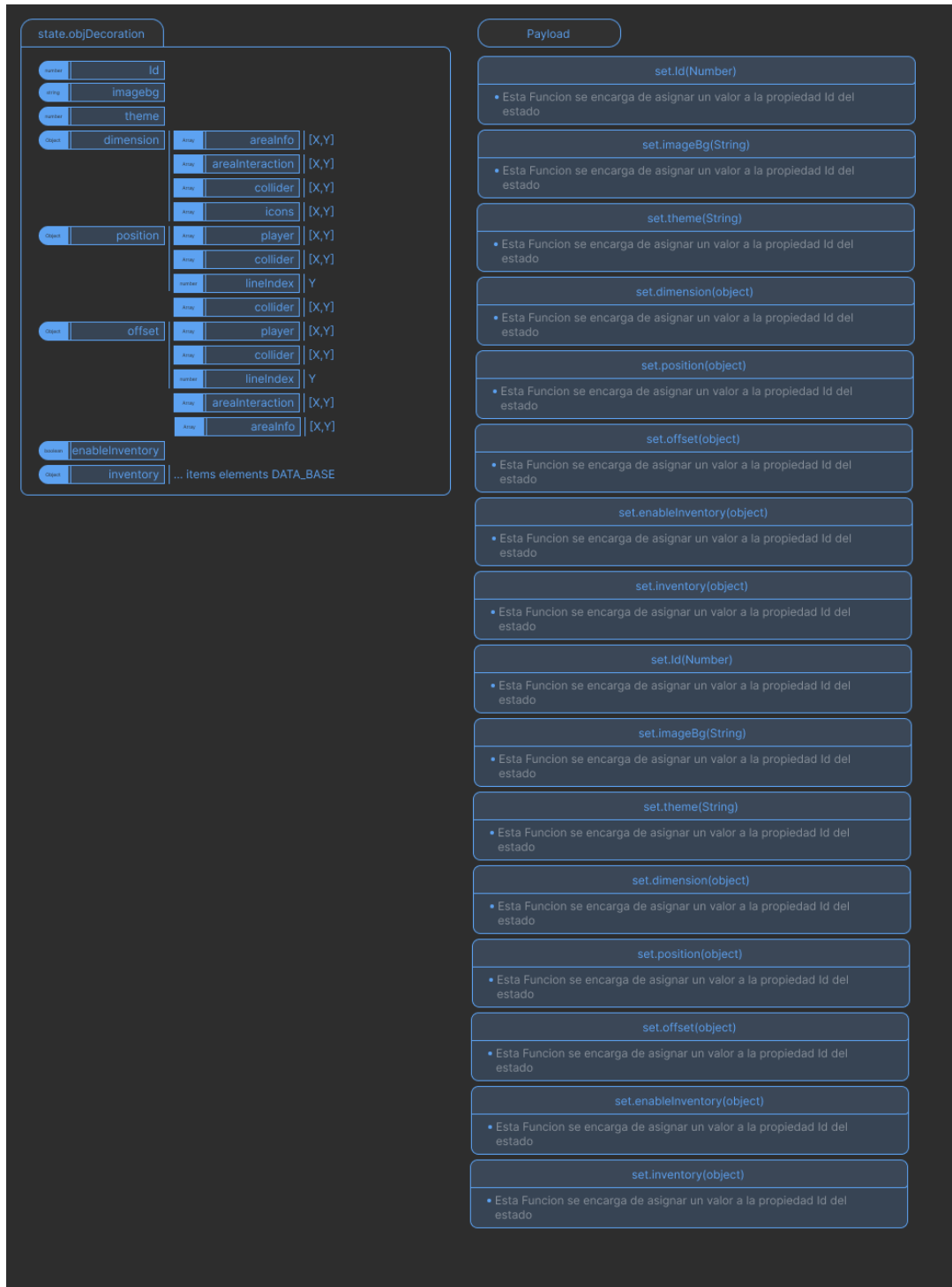


Figura 5: Estructura planeada de `ObjDecoration` y operaciones de actualización.

Las operaciones repetidas que aparecen en el diagrama deben consolidarse en un catálogo único de acciones. Además, `enableInventory` es booleano y no un objeto; la acción correspondiente debería aceptar `bool`. Los nombres `imagebg` y `theme` deberían normalizarse a `imageId` y `themeId` para indicar que son referencias lógicas.

4 Aplicación del flujo en Unity

4.1 Inicialización

La inicialización debe tomar como fuente los componentes existentes de la escena y producir una instantánea válida antes de procesar entrada. El proyecto ya aplica este criterio en `PlayerStateRuntimeApplier.Awake`: obtiene `Rigidbody2D`, `BoxCollider2D`, `WorldYSort` y el rol de la sesión, inicializa el store y aplica el estado.

El orden de ejecución relevante es:

1. `GameSessionStore` se crea o despierta con orden `-1000` y sobrevive cambios de escena.
2. Los objetos de la nueva escena ejecutan `Awake`.
3. `PlayerStateRuntimeApplier`, con orden `-100`, construye el estado inicial del jugador.
4. Los controladores comienzan a leer entrada en `Update` y a mover en `FixedUpdate`.

4.2 Movimiento del jugador

El movimiento implementado usa la física como restricción y el store como fuente lógica de posición:

1. `Update` lee la acción `Player/Move` y limita su magnitud.
2. `FixedUpdate` reconcilia un movimiento físico previo si Unity produjo una posición distinta.
3. Se calcula el desplazamiento solicitado con velocidad y `fixedDeltaTime`.
4. `Physics2D.BoxCast` resuelve cada eje de forma independiente para permitir deslizamiento contra paredes.
5. Se despacha `PlayerAction.SetPosition(allowedPosition)`.
6. El store reduce y publica el nuevo estado.
7. `Rigidbody2D.MovePosition` solicita el movimiento físico.
8. En el siguiente paso fijo se usa `ReconcilePosition` cuando la posición física final difiere de la lógica.

Esta reconciliación evita que el estado quede desfasado cuando otro cuerpo, un contacto o el propio motor ajusta la posición solicitada.

Listing 2: Secuencia esencial implementada para el movimiento

```
1 Vector2 allowedPosition = ResolveAllowedPosition(  
2     store.State.Position,  
3     requestedDisplacement);  
4  
5 store.Dispatch(PlayerAction.SetPosition(allowedPosition));  
6 body.MovePosition(store.State.Position);
```


4.3 Aplicación a componentes de runtime

El `PlayerStateRuntimeApplier` escucha `StateChanged` y refleja en Unity el ordenamiento Y y los datos del collider. La posición se aplica desde el controlador de movimiento, porque debe sincronizarse con `FixedUpdate`. Esta división es válida si se documenta la propiedad de cada efecto:

Cuadro 7: Propietario recomendado de cada efecto

Dato	Aplicador	Motivo
Posición de Rigidbody2D	Controlador de movimiento	Debe respetar el paso fijo y las consultas de colisión.
Tamaño/offset de collider	Runtime applier	Es una proyección directa del estado.
Y-sort	Runtime applier / WorldYSort	Es un efecto visual derivado.
Skin y animación	Adaptador visual del jugador	Aísla Animator, SpriteRenderer y carga de assets.
Decoraciones	World/Decoration runtime registry	Crea, recicla o actualiza vistas según IDs.
Inventario	Adaptador de inventario/UI	La vista reacciona; las reglas permanecen en el dominio.

4.4 Eventos y prevención de ciclos

Un consumidor de `StateChanged` puede aplicar efectos, pero no debería volver a despachar la misma acción sin una condición de salida. Para evitar ciclos:

- el store ignora transiciones cuyo resultado es igual al estado actual;
- las acciones de reconciliación usan tolerancia numérica;
- los aplicadores no leen entrada ni inventan valores de dominio;
- las suscripciones se realizan en `OnEnable` y se eliminan en `OnDisable`.

4.5 Persistencia y cambio de escena

El estado de sesión ya usa `DontDestroyOnLoad`. Para el estado global planeado se recomiendan dos niveles:

- **Estado de sesión:** selección de rol, sala activa y datos necesarios mientras la aplicación está abierta.
- **Estado persistente:** progreso, inventarios y configuraciones que se serializan a JSON o a un repositorio.

No se deben serializar referencias directas a `GameObject`, `Transform`, `Collider2D` o `Sprite`. En su

lugar se guardan IDs y datos simples; al cargar una escena, un registro de runtime resuelve esos IDs a componentes o assets.

5 Ruta de implementación propuesta

La evolución debe ser incremental para conservar el comportamiento actual. Cada etapa puede integrarse y probarse de manera independiente.

5.1 Etapa 1: consolidar contratos y nombres

1. Definir una convención: tipos en `PascalCase`, campos serializados en `camelCase` y acciones con verbo explícito.
2. Resolver términos ambiguos del diagrama: `skinId`, `imageId`, `themeId`, `dimensions`, `colliderSize` y `colliderOffset`.
3. Definir IDs como `string` si pueden provenir de servicios o bases de datos heterogéneas.
4. Crear contratos de inventario, interacción y área antes de agregarlos a los estados principales.

5.2 Etapa 2: completar `PlayerState`

Agregar identidad y apariencia al modelo existente sin reemplazar la ruta de movimiento. Cada dato nuevo requiere, como mínimo:

- campo privado serializado y propiedad de solo lectura;
- parámetro de constructor con valor seguro;
- método `With...`;
- tipo de acción, payload y fábrica de acción;
- caso en el reductor;
- comparación en `StatesAreEqual`;
- prueba del reductor y, si produce un efecto visual, prueba del adaptador.

El correo electrónico debe permanecer en el estado de cuenta o perfil si no es necesario para la simulación del jugador. Separarlo reduce exposición accidental y evita replicar datos privados a otros clientes.

5.3 Etapa 3: implementar `WorldState`

Crear el módulo `World/State` con `WorldStateData`, `WorldAction`, `WorldStateReducer` y `WorldStateStore`. Su primera versión puede manejar mundo, sala, tema y dimensiones. La carga completa debe ser atómica:

Listing 3: Ejemplo de acción compuesta para cargar un mundo

```
1 public static WorldAction LoadWorld(WorldDefinition definition) =>
2     new WorldAction(
3         WorldActionType.LoadWorld,
4         new WorldPayload(
5             definition.Id,
6             definition.DisplayName,
7             definition.RoomId,
8             definition.ThemeId,
9             definition.Dimensions));
```

5.4 Etapa 4: implementar decoraciones como colección

Un único `ObjDecorationStateData` describe una instancia. El store de mundo debería custodiar una colección o un diccionario por ID y ofrecer acciones como:

- `AddDecoration(payload);`
- `RemoveDecoration(id);`
- `MoveDecoration(id, position);`
- `SetDecorationTheme(id, themeId);`
- `SetDecorationInventory(id, inventory).`

Para actualizaciones frecuentes, un diccionario facilita buscar por ID. Para serialización con el `JsonUtility` tradicional de Unity, una lista es más directa; puede mantenerse una lista serializable y construir un índice de runtime no serializado.

5.5 Etapa 5: fachada de estado principal

Solo cuando un caso real lo requiera, crear una fachada que exponga una instantánea coherente de sesión, mundo, jugador y decoraciones. No es obligatorio fusionar todos los stores en un único `MonoBehaviour`. Una fachada de solo lectura puede componerlos:

Listing 4: Fachada conceptual de solo lectura

```
1 public readonly struct PrincipalStateSnapshot
2 {
3     public GameSessionStateData Session { get; }
4     public WorldStateData World { get; }
5     public PlayerStateData Player { get; }
6     public IReadOnlyList<ObjDecorationStateData> Decorations { get; }
7 }
```

Esta opción conserva la independencia de cada ciclo de actualización y permite producir guardados, telemetría o depuración global sin habilitar mutaciones externas.

5.6 Mapa completo de referencia

La Figura 6 reúne los estados y los métodos previstos. Debido a su relación de aspecto, se presenta en una página horizontal para conservar legibilidad.

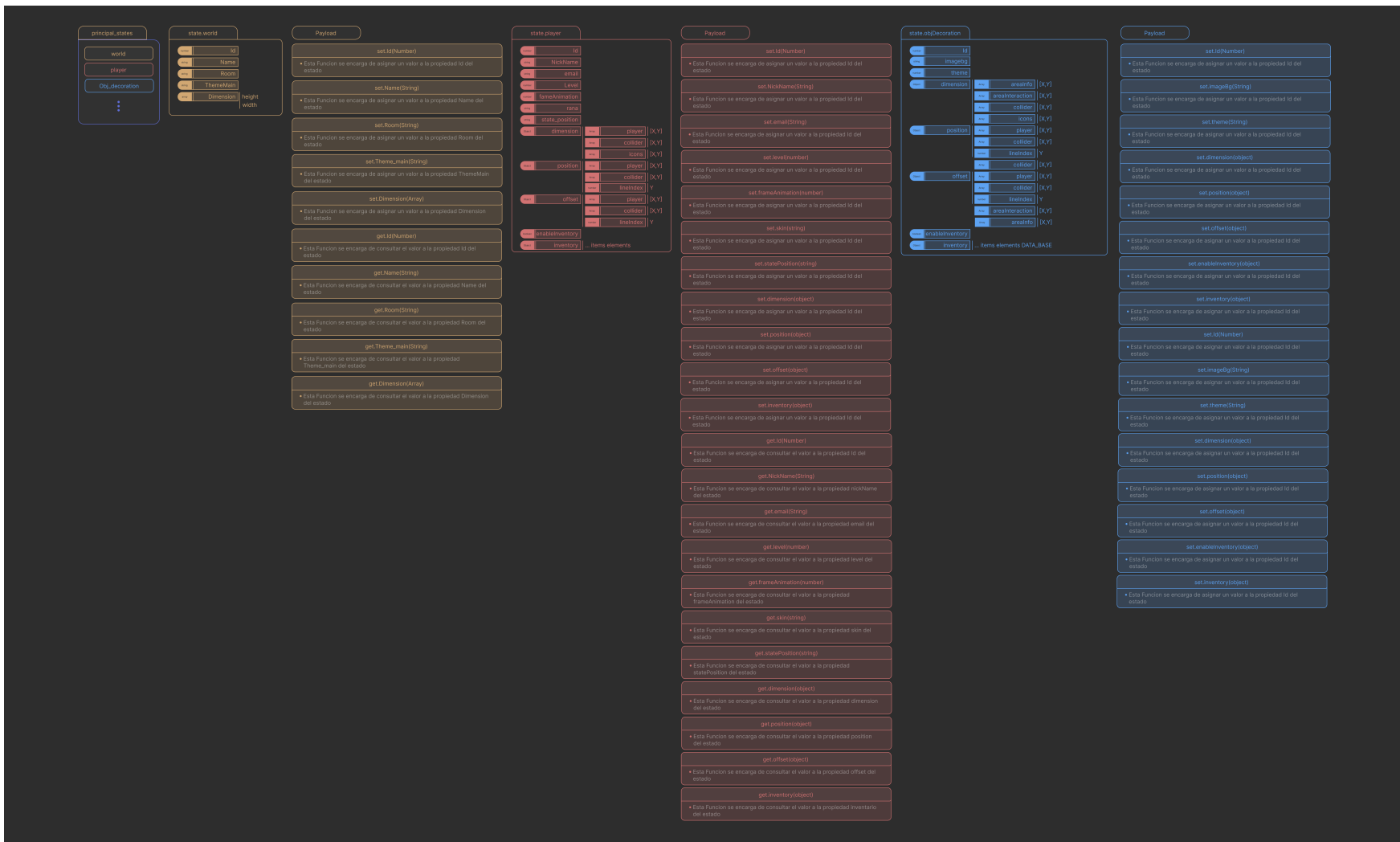


Figura 6: Mapa consolidado de estados, propiedades y operaciones planeadas.

Los métodos del mapa deben interpretarse como inventario de capacidades. En la implementación, los getters son propiedades de solo lectura y los setters se convierten en acciones; no se recomienda copiar literalmente pares `set/get` para cada campo.

6 Validación, pruebas y observabilidad

6.1 Invariantes por dominio

Los reductores deben proteger invariantes baratas y deterministas. Las reglas que necesitan acceso a escena, red o base de datos pertenecen a servicios previos al despacho.

Cuadro 8: Invariantes mínimas recomendadas

Dominio	Invariante	Comportamiento esperado
Player	Cada dimensión del collider es al menos 0.01.	Limitar en el reductor; ya implementado.
Player	El nivel no es negativo.	Limitar o rechazar la acción según la regla de negocio.
Player	La posición contiene valores finitos.	Rechazar NaN e infinito antes de actualizar.
World	Ancho y alto son positivos.	Rechazar mundos sin área válida.
Decoration	El ID no está vacío y no se duplica.	Validar al agregar la instancia.
Decoration	Inventario solo se usa cuando está habilitado.	Vaciar, ignorar o conservar según una política documentada.
Todos	IDs y textos nulos se normalizan.	Convertir a cadena vacía o rechazar según obligatoriedad.

6.2 Pirámide de pruebas

6.2.1 Pruebas unitarias de reductores

Son la base del sistema. Cada tipo de acción debe probar estado inicial, payload, resultado y preservación de campos no relacionados. También deben cubrir valores límite y acciones desconocidas.

6.2.2 Pruebas de stores

Verifican que `StateChanged` se emite una sola vez cuando hay cambio y no se emite para una transición equivalente. Deben comprobar que el evento entrega la acción que causó la transición.

6.2.3 Pruebas de integración en Play Mode

Cubren la conexión con `Rigidbody2D`, `colliders`, `WorldYSort`, animación y cambios de escena. Casos esenciales:

- el jugador no atraviesa obstáculos y puede deslizarse por sus bordes;
- la reconciliación conserva alineados store y cuerpo físico;
- cambiar el tamaño del collider actualiza el componente y respeta el mínimo;
- cambiar de escena preserva la selección de rol;
- crear o quitar una decoración sincroniza el registro visual sin duplicados.

6.3 Registro de transiciones

Durante desarrollo, un observador opcional puede registrar `timestamp`, store, tipo de acción y resumen del estado resultante. No debe incluir correo, tokens ni inventarios sensibles. Para estados grandes se registra un resumen o diferencia, no la instantánea completa en cada frame.

6.4 Rendimiento

El patrón no implica despachar todos los valores visuales continuamente. Recomendaciones:

- no guardar en el estado global cuadros de animación que cambian cada frame, salvo que sean necesarios para red o repetición;
- evitar reconstruir colecciones completas para una sola decoración en escenarios masivos; usar actualización por ID y estructuras adecuadas;
- no publicar eventos si `StatesAreEqual` confirma que no hubo cambio;
- mantener la física en `FixedUpdate` y la lectura de entrada en `Update`;
- perfilar antes de optimizar o reemplazar estructuras claras por soluciones complejas.

6.5 Criterios de aceptación de la arquitectura

La implementación objetivo se considera completa cuando:

1. todo estado se modifica únicamente mediante acciones y reductores;
2. los estados planeados tienen tipos `C#` concretos y serializables;
3. cada campo tiene propietario, significado, unidad y valor inicial documentados;
4. los efectos de Unity están fuera de los reductores;
5. las transiciones críticas cuentan con pruebas unitarias y de integración;
6. el guardado y la carga reconstruyen una instantánea equivalente;
7. los IDs resuelven assets y objetos sin serializar referencias de escena.

7 Anexos

7.1 Correspondencia con el código actual

Las clases citadas se encuentran bajo `Assets/_SocialProyectoCenigraf/Scripts`:

Cuadro 9: Clases implementadas relevantes para la arquitectura

Clase	Responsabilidad observada
<code>GameSessionStateData</code>	Conserva el ID del rol seleccionado.
<code>GameSessionAction</code>	Declara selección y limpieza de rol con payload tipado.
<code>GameSessionReducer</code>	Produce la nueva sesión de forma determinista.
<code>GameSessionStore</code>	Store singleton persistente entre escenas.
<code>PlayerStateData</code>	Conserva posición, rol, Y-sort, tamaño y offset del collider.
<code>PlayerAction</code>	Declara posición, traslación, reconciliación, rol, Y-sort y collider.
<code>PlayerStateReducer</code>	Calcula el siguiente estado y limita dimensiones de collider.
<code>PlayerStateStore</code>	Custodia el estado del jugador y publica cambios reales.
<code>PlayerStateRuntimeApplier</code>	Inicializa y aplica collider y Y-sort al runtime.
<code>PlayerMovementController</code>	Lee input, resuelve colisiones, despacha posición y reconcilia física.
<code>CameraStateData</code>	Conserva estado lógico de posición, seguimiento y zoom de cámara.
<code>CameraAction/Reducer/Store</code>	Implementan el mismo flujo unidireccional para cámara.
<code>WorldYSort</code>	Deriva <code>sortingOrder</code> desde la coordenada Y.

7.2 Glosario

Acción	Mensaje inmutable que identifica una intención de cambio.
Payload	Datos específicos transportados por una acción.
Reductor	Función determinista que calcula el siguiente estado.
Store	Componente propietario de una instantánea y del método de despacho.
Instantánea	Valor completo del estado en un momento determinado.
Reconciliación	Corrección del estado lógico con el resultado definitivo del runtime o servidor.

Runtime applier

Adaptador que traduce datos del estado a componentes Unity.

Y-sort

Ordenamiento visual basado en la posición vertical para simular profundidad 2D.

7.3 Decisiones pendientes

Antes de implementar todos los diagramas, el equipo debe resolver:

- si el ID canónico es texto o número en los servicios externos;
- qué representa exactamente cada matriz $[X,Y]$ del diseño;
- si `frameAnimation` es persistente, replicado o exclusivamente visual;
- si el inventario vive en jugador/decoración o en un store normalizado por propietario;
- qué datos del jugador son privados y cuáles pueden compartirse con otros clientes;
- si la sala equivale a una escena Unity, una zona dentro de escena o una sesión de red;
- si el estado principal será un store único o una fachada de stores por dominio.

7.4 Registro de fuentes visuales

Los seis diagramas suministrados se copiaron a `_Doc/images` con nombres normalizados. Todas las figuras forman parte del proyecto documental y usan rutas relativas, por lo que no dependen de la carpeta de descargas del autor.

Cuadro 10: Archivos visuales incorporados

Archivo	Uso
<code>composicion-estados.png</code>	Vista general de subestados y campos.
<code>flujo-datos.png</code>	Flujo unidireccional y ejemplo de movimiento.
<code>mapa-estados-metodos.png</code>	Mapa consolidado de propiedades y operaciones.
<code>estructura-world.png</code>	Detalle de World y su payload conceptual.
<code>estructura-player.png</code>	Detalle de Player y sus operaciones.
<code>estructura-obj-decoration.png</code>	Detalle de ObjDecoration y sus operaciones.